

Output1:

```
abhi@rajubhai17: ~/myParalle X + v
abhi@rajubhai17:~/myParallel$ ls
prog1  program1.c
abhi@rajubhai17:~/myParallel$ gcc -fopenmp program1.c -o prog1
abhi@rajubhai17:~/myParallel$ export OMP_NUM_THREADS=4
abhi@rajubhai17:~/myParallel$ ./prog1
sequential Merge Sort Time: 0.016312 seconds
parallel Merge sort time: 0.013877 seconds
abhi@rajubhai17:~/myParallel$
```

Output2:

```
abhi@rajubhai17: ~/myParalle X + v - □ X
abhi@rajubhai17:~/myParallel$ gcc -fopenmp program2.c -o prog2
abhi@rajubhai17:~/myParallel$ export OMP_NUM_THREADS=4 && ./prog2
Enter number of iterations: 8
Thread 0 : Iterations 0 -- 1
Thread 1 : Iterations 2 -- 3
Thread 2 : Iterations 4 -- 5
Thread 3 : Iterations 6 -- 7
abhi@rajubhai17:~/myParallel$
```

Output3:

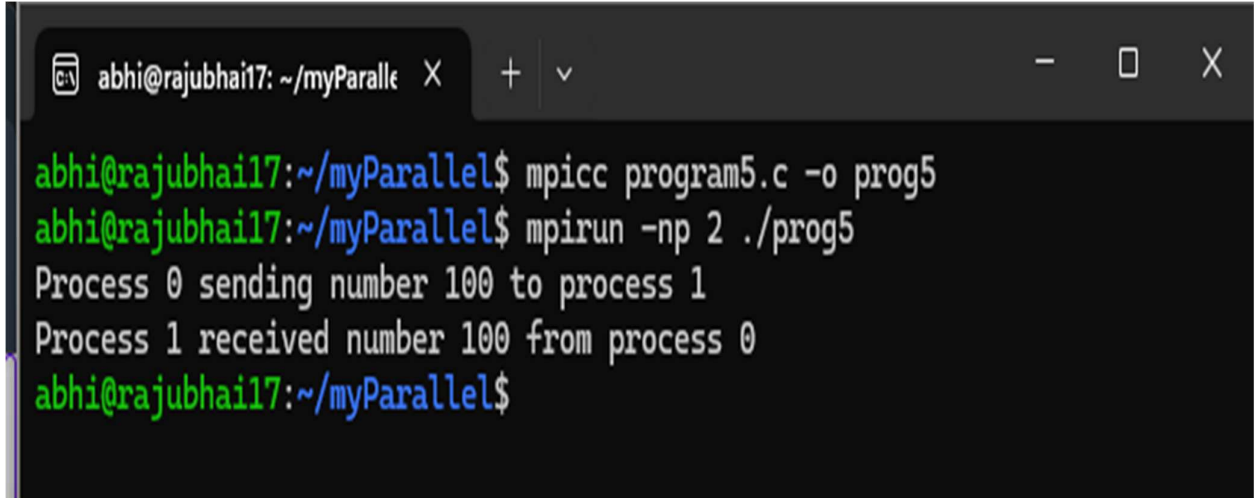
```
abhi@rajubhai17: ~/myParallel X + v - □  
abhi@rajubhai17:~/myParallel$ gcc -fopenmp program3.c -o prog3  
abhi@rajubhai17:~/myParallel$ export OMP_NUM_THREADS=4 && ./prog3  
Enter number of Fibonacci terms: 15  
Fibonacci Series:  
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377  
abhi@rajubhai17:~/myParallel$
```

Output4:

```
abhi@rajubhai17: ~/myParalle X + v - □ X
abhi@rajubhai17:~/myParallel$ gcc -fopenmp program4.c -o prog4 -lm
abhi@rajubhai17:~/myParallel$ export OMP_NUM_THREADS=4 && ./prog4
Enter the value of n: 100

prime numbers from 1 to 100:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
serial Times: 0.000005 seconds
parallel Time: 0.000004 seconds
abhi@rajubhai17:~/myParallel$
```

Output5:

A terminal window with a dark background and light-colored text. The window title bar shows 'abhi@rajubhai17: ~/myParalle' and standard window controls. The terminal content shows the compilation of 'program5.c' to 'prog5' using 'mpicc', followed by its execution with 'mpirun -np 2'. The output shows two processes: Process 0 sending the number 100 to Process 1, and Process 1 receiving it. The prompt returns to the user.

```
abhi@rajubhai17:~/myParallel$ mpicc program5.c -o prog5
abhi@rajubhai17:~/myParallel$ mpirun -np 2 ./prog5
Process 0 sending number 100 to process 1
Process 1 received number 100 from process 0
abhi@rajubhai17:~/myParallel$
```

Output6:

```
abhi@rajubhai17: ~/myParallel X + v - □ X
abhi@rajubhai17:~/myParallel$ mpicc program6.c -o prog6
abhi@rajubhai17:~/myParallel$ mpirun -np 2 ./prog6
process 0 received back number: 123
Process 1 received and sent back number: 123
  abhi@rajubhai17:~/myParallel$ mpicc program6.c -o prog6
abhi@rajubhai17:~/myParallel$ mpirun -np 2 ./prog6
process 0 waiting to receive from process 1..
process 1 waiting to receive from Process 0...
```

Output7:

```
abhi@rajubhai17: ~/myParallel X + v - □ X
abhi@rajubhai17:~/myParallel$ mpicc program7.c -o prog7
abhi@rajubhai17:~/myParallel$ mpirun -np 4 ./prog7
Process 0 broadcasting number 50 to all other processes.
Process 2 received number 50
Process 3 received number 50
Process 0 received number 50
Process 1 received number 50
abhi@rajubhai17:~/myParallel$ |
```

Output8:

```
abhi@rajubhai17: ~/myParallel X + v - □ X
abhi@rajubhai17:~/myParallel$ mpicc program8.c -o prog8
abhi@rajubhai17:~/myParallel$ mpirun -np 4 ./prog8
Gathered data in root process:
gathered_data[0]=0
gathered_data[1]=11
gathered_data[2]=22
gathered_data[3]=33
abhi@rajubhai17:~/myParallel$ |
```


Output9:

```
abhi@rajubhai17: ~/myParalle X + v - □ X
abhi@rajubhai17:~/myParallel$ mpicc program9.c -o prog9
abhi@rajubhai17:~/myParallel$ mpirun -np 4 ./prog9
Result using MPI_Reduce(only root printd)--
sum=10
Product=24
Max= 4
Min= 1
Process 0 has value 1
Process 0 sees(Mpi_Allreduce):Sum=10 ,Prod=24 ,Max=4 ,Min=1
Process 1 has value 2
Process 1 sees(Mpi_Allreduce):Sum=10 ,Prod=24 ,Max=4 ,Min=1
Process 2 has value 3
Process 2 sees(Mpi_Allreduce):Sum=10 ,Prod=24 ,Max=4 ,Min=1
Process 3 has value 4
Process 3 sees(Mpi_Allreduce):Sum=10 ,Prod=24 ,Max=4 ,Min=1
abhi@rajubhai17:~/myParallel$
```